CSS4003 — DevOps 2

Lecture 7: AWS EC2

Nauruzbayeva Farikha · Lesson 7

Recap — Lesson 6 (Introduction to Cloud & AWS)

- Region vs Availability Zone — what's the difference? (a region is a geographic area like `eu-central-1`; an AZ is one or more isolated data centers inside it — a region has several)
- On EC2, who patches the operating system — AWS or you? (you — on IaaS the guest OS is the customer's side of the shared responsibility model)
- Your $1 budget alert fires. Does AWS stop your resources? (no — a budget only alerts; stopping things is your job)
- `aws sts get-caller-identity --profile css4003` shows `...:root`. What's wrong? (you configured root access keys — delete them and use your IAM admin user instead)

Today’s agenda

- ☐ What an EC2 instance is, and the AMI it starts from
- ☐ Instance types, and how to read a name like `m7g.large`
- ☐ Pricing models: on-demand, Savings Plans, Spot
- ☐ Connecting: key pairs and SSH, Instance Connect, Session Manager
- ☐ Security groups, public vs private IPs, Elastic IPs
- ☐ User data: a server that configures itself on first boot
- ☐ Lifecycle: stop vs terminate, what survives, metadata & roles
- ☐ Launching from the CLI, monitoring, and cleaning up

The scenario

Last week you secured an AWS account. Now the team wants the app on a real server. Someone launches an instance from the console, opens port 22 to the whole internet "just for now", installs nginx by hand over SSH, and stops the instance on Friday to save money. On Monday the public IP is different and nobody remembers which packages they installed.

Today: launch an EC2 server you can reach safely, rebuild from one command, and shut down without surprises on the bill.

Block 1

Instances and AMIs

An EC2 instance is a rented virtual machine

- **EC2** = Elastic Compute Cloud. You pick CPU, memory, disk and OS, and AWS runs the VM on its hardware in the AZ you choose.
- It behaves like any Linux (or Windows) server: SSH in, install packages, run services.
- It lives inside a **VPC subnet** (week 8). New accounts come with a **default VPC** in every region, so you can launch right away.
- Billed while it's running. Linux instances are billed **per second**, with a 60-second minimum.

Shared responsibility from last week: AWS runs the host and hypervisor. Everything from the OS up — patches, firewall rules, your app — is yours.

AMI — the image an instance boots from

- An **AMI** (Amazon Machine Image) is a template: OS, preinstalled software, and the root disk snapshot.
- Common choices in this course:
  - **Amazon Linux 2023** — AWS's own distro, `dnf` package manager, default user `ec2-user`
  - **Ubuntu Server LTS** — `apt`, default user `ubuntu`
- AMIs are **regional** and have IDs like `ami-0abc...`. The same OS has a different AMI ID in every region, and new IDs appear with every update.
- You can make your own AMI from a configured instance ("golden image").

Don’t hard-code AMI IDs

```
# AWS publishes the latest AMI IDs as public SSM parameters
aws ssm get-parameter \
  --name /aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64 \
  --query Parameter.Value --output text
# ami-0...   (different per region, changes over time)
```

- An AMI ID copied from a blog post is probably for another region, or out of date.
- Look it up at launch time instead. `run-instances` can even resolve the parameter itself — we’ll use that in Block 5.

Instance types: how to read the name

`m7g.large`

Part	Meaning	Examples
family	What it's optimized for	`t` burstable, `m` general, `c` compute, `r` memory
generation	Higher = newer hardware	`t2` → `t3`, `m5` → `m7`
attributes	CPU or extra features	`g` Graviton (ARM), `a` AMD, `i` Intel, `d` local NVMe disk
size	How much CPU / memory	`nano`, `micro`, `small`, `medium`, `large`, `xlarge`, `2xlarge` …

Each size up roughly doubles vCPUs and memory, and the price with it. Check exact specs in the console or with `aws ec2 describe-instance-types`.

Burstable instances: `t3`, `t4g`

- `t` instances have a low **baseline** CPU and earn **CPU credits** while they're idle. They spend credits to burst above the baseline.
- Perfect for small web apps, dev boxes, this course: mostly idle, busy now and then.
- `t3` and `t4g` launch in **unlimited** mode by default: if credits run out, they keep bursting and you pay for the extra CPU.
- Running a CPU-heavy job on a `t` instance all day? Pick `m` or `c` instead.

`g` types (Graviton, ARM) are usually cheaper for the same work, but you need an **arm64** AMI, and your software must support ARM.

Pricing models

Model	You commit to	Good for	Catch
On-demand	Nothing	Learning, unpredictable load	Highest price per hour
Savings Plans / Reserved Instances	1 or 3 years of usage	Steady, always-on servers	You pay even if you stop using it
Spot	Nothing	Batch jobs, CI runners, anything restartable	AWS can reclaim it with a **2-minute** warning

Discounts for commitments and Spot are large, but they vary by type, region and time. Compare current numbers in the Pricing Calculator, not on slides. In this course: **on-demand only**, and turn things off.

Quick check

Your team's CI pipeline runs test jobs that take 10 minutes each and can be retried. Which pricing model fits, and why?

Spot — the jobs are short and restartable, so a 2-minute interruption just means a retry, and Spot is much cheaper than on-demand.

Block 2

Connecting to your instance

Key pairs

- A **key pair** = a public key AWS puts on the instance at launch, plus a private key (`.pem`) that only you have.
- AWS shows you the private key **once**, at creation. Lose it, and you can't SSH with that key anymore.
- Linux instances use the key for SSH login. There is no password login by default.

```
aws ec2 create-key-pair --key-name css4003-key \
  --query KeyMaterial --output text > css4003-key.pem
chmod 400 css4003-key.pem        # SSH refuses keys that others can read
```

A `.pem` file is a password. Never commit it — add `*.pem` to `.gitignore` .

SSH in

```
ssh -i css4003-key.pem ec2-user@<public-ip>      # Amazon Linux 2023
ssh -i css4003-key.pem ubuntu@<public-ip>        # Ubuntu
```

- The username depends on the AMI, not on you: `ec2-user`, `ubuntu`, …
- First connection asks you to confirm the host fingerprint — type `yes`.
- `Permission denied (publickey)` → wrong user, wrong key, or key file permissions too open.
- `Connection timed out` → the network blocks you: security group, no public IP, or wrong address.

Windows: OpenSSH is built into Windows 10/11 — the same `ssh` command works in PowerShell.

Keyless options

EC2 Instance Connect

- "Connect" button in the console, opens SSH in the browser
- Pushes a **temporary** key for a short time
- Still needs port **22** open to the Instance Connect service, and the agent (preinstalled on Amazon Linux 2023 and recent Ubuntu)

Systems Manager Session Manager

- Shell in the browser or via `aws ssm start-session`
- **No inbound port at all**, no key pair
- Needs the SSM agent (preinstalled on Amazon Linux 2023) and an instance role with `AmazonSSMManagedInstanceCore`
- Every session can be logged

In production, teams often close port 22 entirely and use Session Manager. For learning, SSH with a key pair is fine — just restrict who can reach it.

Block 3

Security groups & IP addresses

Security groups — the instance firewall

- A **security group** (SG) is a set of **allow** rules attached to an instance's network interface.
- **Inbound:** everything is blocked unless a rule allows it.
- **Outbound:** everything is allowed by default.
- **Stateful:** if a request is allowed in, the reply is allowed out automatically.
- No "deny" rules. You can only allow.
- Change rules any time — they apply immediately, no reboot.

A sane web-server security group

Direction	Port	Source	Why
Inbound	22 (SSH)	**your IP** `/32`	Only you can try to log in
Inbound	80 (HTTP)	`0.0.0.0/0`	Anyone can see the website
Inbound	443 (HTTPS)	`0.0.0.0/0`	Same, encrypted
Outbound	all	`0.0.0.0/0`	Updates, package downloads

22 from 0.0.0.0/0 = the whole internet can try to log in. Bots scan for open SSH constantly. "Just for now" usually becomes "forever".

Security groups from the CLI

```
SG=$(aws ec2 create-security-group --group-name web-sg \
  --description "SSH from me, HTTP from anyone" \
  --query GroupId --output text)
MYIP=$(curl -s https://checkip.amazonaws.com)
aws ec2 authorize-security-group-ingress --group-id $SG --protocol tcp --port 22 --cidr $MYIP/32
aws ec2 authorize-security-group-ingress --group-id $SG --protocol tcp --port 80 --cidr 0.0.0.0/0
aws ec2 describe-security-groups --group-ids $SG
```

Without `--vpc-id`, the group goes into the region's default VPC. Your home or university IP can change — if SSH suddenly times out, check it first.

Public IP, private IP, Elastic IP

- **Private IP** — inside the VPC (e.g. `172.31.x.x` in a default VPC). Kept for the instance's whole life, including stop/start.
- **Public IP** — auto-assigned from AWS's pool, so the internet can reach you. **Released on stop**; you get a new one on start.
- **Elastic IP** — a static public IPv4 you allocate and attach. Survives stop/start; you can move it to another instance.
- AWS charges for **every public IPv4 address**, auto-assigned or Elastic, attached or idle. Release Elastic IPs you're not using.

The scenario's "Monday surprise": the instance was stopped, so its auto-assigned public IP changed. Fix with an Elastic IP or DNS.

Quick check

SSH to your new instance hangs and then says `Connection timed out`. Name two likely causes.

The security group doesn't allow port 22 from your current IP, or the instance has no public IP (or you're using an old one). A timeout is a network problem; a wrong key gives `Permission denied` instead.

Block 4

User data: servers that configure themselves

User data

- **User data** is a script (or cloud-init config) you pass at launch.
- **cloud-init** runs it **once, as root, on the first boot**.
- Use it to install packages, write config files, start services.
- No manual SSH steps → every new instance comes up identical.
- Logs: `/var/log/cloud-init-output.log` on the instance.

This is the first step from "pet servers" you fix by hand to "cattle" you rebuild from code. Terraform in week 14 takes it further.

Example: nginx on first boot (Amazon Linux 2023)

```
#!/bin/bash
dnf install -y nginx
echo "<h1>CSS4003 — hello from $(hostname -f)</h1>" > /usr/share/nginx/html/index.html
systemctl enable --now nginx
```

- Save as `user-data.sh`, pass it at launch, open `http://<public-ip>` a minute or two later.
- On Ubuntu: `apt-get update && apt-get install -y nginx`, and the web root is `/var/www/html`.
- Changing user data on an existing instance does **not** re-run it. To apply changes, launch a new instance.

When user data "didn’t work"

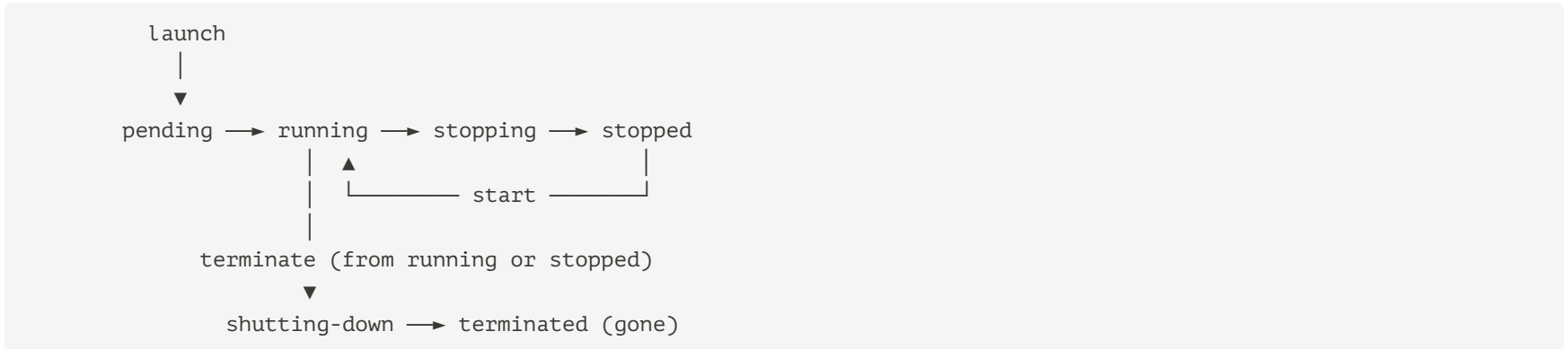
Symptom	Check
Page doesn’t load at all	Security group allows 80? Using `http://`, not `https://`?
Page loads default nginx page	Script failed after install — read `/var/log/cloud-init-output.log`
Nothing installed	First line must be `#!/bin/bash`; package name right for this distro?
Worked once, not after edits	User data only runs on first boot — launch a fresh instance

Give it a minute or two. The instance shows "running" before cloud-init has finished installing packages.

Block 5

Lifecycle, metadata, and launching from the CLI

Instance lifecycle



Reboot keeps the instance in "running": same host, same public IP, same disks.

Stop vs terminate: what happens to the disks?

	Reboot	Stop → start	Terminate
Root EBS disk	Kept	Kept	**Deleted** (DeleteOnTermination = true by default)
Extra EBS volumes	Kept	Kept	Depends on each volume's DeleteOnTermination flag
Instance store (local NVMe)	Kept	**Lost**	**Lost**

Anything you care about on instance store is gone after a stop. Keep real data on EBS, S3 or a database.

Stop vs terminate: addresses and billing

	Reboot	Stop → start	Terminate
Private IP	Kept	Kept	Gone
Auto-assigned public IP	Kept	**New one**	Gone
Elastic IP	Kept	Kept	Detached — still billed until you release it
Compute billing	Continues	Stops	Stops

Stopped is not free: the EBS disks (and any Elastic IP) still bill.

Termination protection

- `terminate` is final. There is no recycle bin.
- **Termination protection** blocks terminate from the console, CLI and API until you turn it off.
- Turn it on for anything you'd hate to lose:

```
aws ec2 modify-instance-attribute --instance-id i-0123456789abcdef0 \
  --disable-api-termination
```

Better still: keep the data off the instance (S3, RDS, a separate EBS volume) and make the instance itself rebuildable from user data.

Instance metadata (IMDSv2)

```
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 300")
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/instance-id
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/placement/availability-zone
```

- From **inside** the instance, `169.254.169.254` answers "who am I?": instance ID, AZ, IPs, and the instance role's temporary credentials.
- **IMDSv2** requires a session token first (the `PUT`). That blocks a class of attacks where a vulnerable app is tricked into fetching URLs.
- Amazon Linux 2023 AMIs require IMDSv2 by default. Keep it that way.

Instance profiles: credentials without keys

- Your app on EC2 needs to read S3. **Don't** copy access keys onto the server.
- Attach an **IAM role** to the instance (an *instance profile*). The AWS CLI and SDKs pick up **temporary credentials** from instance metadata automatically and AWS rotates them.
- On the instance, `aws sts get-caller-identity` shows `assumed-role/<role-name>/i-...`, with no `~/.aws/credentials` file.

Rule: people use access keys (with MFA); servers use roles. More on roles when we deploy to AWS in week 13.

Launch from the CLI

```
aws ec2 run-instances \
  --image-id resolve:ssm:/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64 \
  --instance-type t3.micro \
  --key-name css4003-key \
  --security-group-ids $SG \
  --user-data file://user-data.sh \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=css4003-web}]' \
  --query 'Instances[0].InstanceId' --output text
```

`resolve:ssm:` looks up the latest Amazon Linux 2023 AMI for your region at launch. Check the free-tier-eligible instance types for your account on `aws.amazon.com/free` before picking one.

Then find it, use it, and remove it

```
IID=i-0123456789abcdef0                       # from run-instances
aws ec2 wait instance-running --instance-ids $IID
aws ec2 describe-instances --instance-ids $IID \
  --query 'Reservations[0].Instances[0].[State.Name,PublicIpAddress]' --output text
curl http://<public-ip>                       # nginx page from user data

aws ec2 stop-instances --instance-ids $IID      # pause: disk still bills
aws ec2 terminate-instances --instance-ids $IID # done: gone for good
```

The console's "Launch instance" wizard asks for exactly the same things: name/tags, AMI, type, key pair, network + security group, storage, and user data under "Advanced details".

Monitoring basics: status checks

- **System status check** — AWS's side: the host, power, network. If it fails, stop and start the instance so it moves to a healthy host.
- **Instance status check** — your side: the OS didn't boot, out of memory, broken network config, a bad `/etc/fstab`.
- The console also shows basic **CloudWatch metrics**: CPU, network, disk I/O. Memory and disk-space usage need the CloudWatch agent.
- Can't SSH in? **Actions → Monitor and troubleshoot → Get system log** shows the boot console output.

Quick check

You stopped your instance over the weekend to save money. List everything that is still on your bill.

The EBS volumes (root and any extras), any Elastic IP, and any snapshots. Compute stops; storage and reserved addresses don't.

Block 6

Cost hygiene & try it yourself

End-of-session checklist

1. **Terminate** instances you don’t need next week. Stop the ones you do.
2. **Release** Elastic IPs you allocated.
3. Delete leftover **volumes** in the "available" state and old snapshots.
4. Check the **region selector** — and every region you touched.
5. Glance at **Billing → Bills** for this month.

Free-tier rules changed for accounts created after July 15, 2025, and they differ by account. "It was free tier" is not a defense against a bill — read the current terms on `aws.amazon.com/free`.

Try it yourself

With an AWS account

1. Key pair + `chmod 400`
2. Security group: 22 from your IP, 80 from anywhere
3. `run-instances` with the nginx user data
4. Open the page, SSH in, read the cloud-init log
5. Stop → start: compare the public IP
6. Terminate, and check nothing is left

Without AWS

Same idea on a local VM with **Multipass**, which also uses cloud-init:

```
#cloud-config
packages: [nginx]
runcmd:
  - systemctl enable --now nginx
```

```
multipass launch --name web --cloud-init web.yaml
multipass info web       # get its IP
curl http://<ip>
multipass delete --purge web
```

By the end of this lesson, you should be able to

- ☐ Explain what an AMI and an instance type are, and read a type name
- ☐ Choose between on-demand, Savings Plans and Spot for a workload
- ☐ Connect to an instance with SSH, and name two keyless alternatives
- ☐ Write a security group that allows SSH only from your IP
- ☐ Bootstrap a server with user data and debug it from the cloud-init log
- ☐ Say what survives stop, start and terminate — and what keeps billing
- ☐ Launch, inspect and terminate an instance from the AWS CLI

Before next lecture

- ☐ Terminate (or at least stop) every instance from today
- ☐ Release any Elastic IP you allocated
- ☐ Look up your default VPC in the console: its subnets and route table

Next week we open up that default VPC and build our own network around the instance.

Next lecture

AWS Networking — VPCs, subnets, route tables, internet gateways, and where your EC2 instance actually lives.